Neural Networks of GAN Style for Poisson Equation

Qimu Luo

November 12, 2023

Contents

1 Introduction **1**

2 Therory **2**
2.1 Illustration 2
2.2 Framework of Algorithm 3

3 Python Code Implementation **4**
3.1 Interpretation by Modules 4
3.1.1 Set random seeds 4
3.1.2 Building 2 neural networks 4
3.1.3 Creat a class to solve PDE 6
3.1.4 Create a class for the example 9
3.1.5 Training and Drawing 10

4 Analysis of The Result **12**

5 Reflection of Code **13**

6 Conclusion **15**

1 Introduction

This report describes a method inspired by a journal (Deep learning for Partial Differential Equations (PDEs)), in which the authors used two neural networks to approximate boundary and interior conditions for solving a partial differential equation (Poisson Equation). The reference solution was computed on a 50×50 grid considering a unit square $\Omega = [0, 1]^2$, with a test on a well-behaved solution $u(x, y) = \sin \pi x \sin \pi y$.

2 Therory

2.1 Illustration

1. The goal: The goal is to solve the Poisson equation u, which is a specific case of elliptic partial differential equations:

$$\begin{cases} \Delta u = f & \text{in} \quad \Omega \\ u = g_D & \text{on} \quad \partial\Omega \end{cases} \tag{1}$$

   - Δ: Laplace operator, f represents the Laplacian of u

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \tag{2}$$

   - Ω: Inner domain
   - $\partial\Omega$: Boundary condition

2. Core method: The core algorithm of for solving equation 1 is as follows: approximate u with

$$u(x, y; w_1, w_2) = A(x, y; w_1) + B(x, y) \cdot N(x, y; w_2) \tag{3}$$

   - $A(x, y; w_1)$: Boundary network approximates the boundary condition:

$$A(x, y; w_1)|_{\partial\Omega} \approx g_D \tag{4}$$

   - $B(x, y)$: A switch function, satisfying $B(x, y)|_{\partial\Omega} \equiv 0$:

$$B(x, y) = x \cdot (1 - x) \cdot y \cdot (1 - y) \tag{5}$$

   - $N(x, y; w_2)$: PDE network approximates the inner domain condition, which is coupled with $A(x, y; w_1)$

3. Evaluation indicators: Three error metrics are utilized to evaluate the effectiveness of the method and MSE (Mean Squared Loss) is used for the loss function:

$$\text{Bounding loss} \quad L_b = \frac{1}{n_b} \sum_{i=1}^{n_b} (A(x_i, y_i; w_i) - u_i)^2 \tag{6}$$

$$\text{PDE loss} \quad L_p = \frac{1}{n_p} \sum_{i=1}^{n_p} (\Delta u_h(x_i, y_i; w_1, w_2) - f_i)^2 \tag{7}$$

$$L_2 \text{ error} \quad L_2 = \sqrt{\frac{1}{m} \sum_{i=1}^{m} |(u_h(x_i, y_i; w_1, w_2) - u(x_i, y_i)|^2} \tag{8}$$

   Note: The losses 6 and 7 are utilized for training, while 8 is used to demonstrate the difference between the exact and numerical solutions produced by neural networks.

4. Training algorithm: The training algorithm for the two neural networks was based on GAN (Generative Adversarial Neural Network) style, where the Boundary network is updated multiple times (k) before the PDE network can be updated once in each iteration. In other words, the boundary network is considered a generator while the PDE network is the discriminator. Both the boundary network and the PDE network are trained simultaneously based on the basic idea of GAN.

The figure 1 illustrates the whole algorithm.

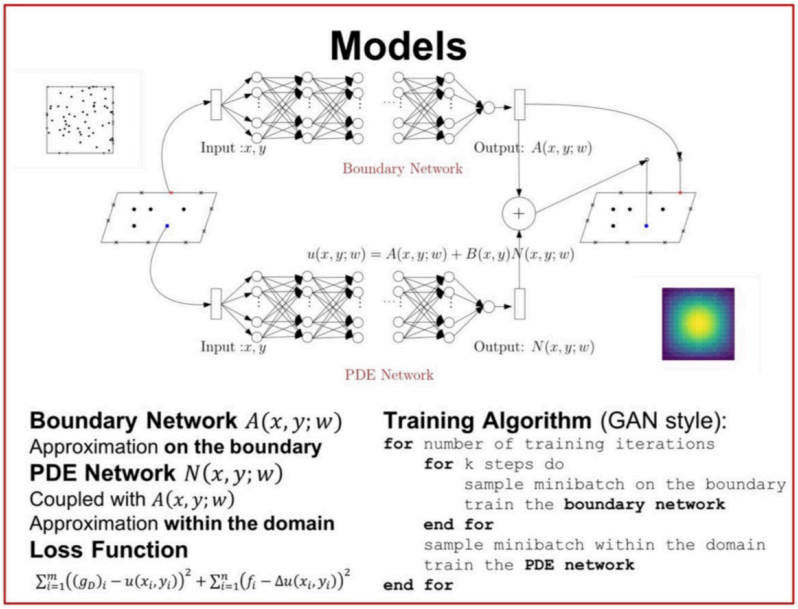


Figure 1: Models

Note: There are two discrepancies between my method with the figure 1 provided by the journal.

1. Loss function: The loss function is composed of 6 and 7 using the MSE, instead of the square sum of the loss.

2. Training data: Since the overall data set is small, full-batch training data is generated randomly from the inner domain and boundary for training, instead of randomly sampling mini-batches for each update.

2.2 Framework of Algorithm

The code was divided into five parts to enhance code reusability: network structure, PDE solver class, example class, training, and visualization functions.

1. Network structure: PyTorch was used to create the structure of both the Boundary and PDE networks, which will be used to input data. Each network has 3 hidden layers with 64 neurons in each layer.

2. PDE solver: The class covers the fundamental items required to solve a partial differential equation.
    - Define hyperparameters in the initialize function.
    - Prepare the exact solution and right-hand side function (to be modified).
    - Prepare data for the boundary, inner domain, and reference points on the 50×50 grid.
    - Prepare loss: boundary loss, PDE loss, L_2 error.
3. Example (well-behaved): This class inherits methods from parent class PDE solver and populates its exact solution and right-hand side function.
4. Training: The function is utilized to train neural networks and record loss.
    - Initialize 3 empty lists to store the loss values later.
    - Train boundary network for k steps in each iteration while utilizing an early termination strategy to maintain boundary loss around $\epsilon = 10^{-5}$.
    - Train the PDE network in each iteration, and record both PDE loss and L_2 error.
    - Return the trained neural network and the 3 metrics for visualizing purposes.
5. Visualization: Aim to compare numerical solutions by neural networks and manufactured solutions, and visualize the three losses in each iteration.

3 Python Code Implementation

3.1 Interpretation by Modules

3.1.1 Set random seeds

To ensure consistent results, a specific seed value is used to reproduce the same set of random numbers.

```
    def setup_seed(seed):
    torch.manual_seed(seed)
    # Results of random operations on the CPU will be deterministic
       and reproducible.
setup_seed(888888) # Choose a value large enough to ensure randomness
```

3.1.2 Building 2 neural networks

Networks are structured through a series of steps as follows:

1. Defines boundary network class that inherits from the 'torch.nn.Module' class.

2. Initialization: where 'super() init()' line calls the initialization method of the parent class (torch.nn.Module) for proper initialization of the boundarynet class.

3. Design my structure:

   - 'torch.nn.Sequential': Allows for the stacking of multiple layers sequentially.
   - Layers: First layer (2×64)– two-dimensional since the input consists of coordinates (x, y); Through 3 hidden layers, each with 64 neurons, the network processes the input data; The last layer (64×1)– outputs a scalar value.
   - 'torch.nn.Tanh()': Tanh activation function is utilized on the outputs of each layer to squash the value betweeen -1 and 1.
   - 'forward(self, x)': Define the forward process of FNN.
   - boundaryN: Create a instance for training later.

```
class boundarynet(torch.nn.Module):
def __init__(self):
super().__init__()#Call the initialization of torch.nn.Module

#define my own net
self.net = torch.nn.Sequential(
torch.nn.Linear(2, 64),
torch.nn.Tanh(),
torch.nn.Linear(64, 64),
torch.nn.Tanh(),
torch.nn.Linear(64, 64),
torch.nn.Tanh(),
torch.nn.Linear(64, 1),
)
def forward(self, x):
return self.net(x)

boundaryN = boundarynet()#intance

class pdenet(torch.nn.Module):
def __init__(self):
super().__init__()
self.net = torch.nn.Sequential(
torch.nn.Linear(2, 64),
torch.nn.Tanh(),
torch.nn.Linear(64, 64),
torch.nn.Tanh(),
torch.nn.Linear(64, 64),
```

```
    torch.nn.Tanh(),
    torch.nn.Linear(64, 1),
    )
    def forward(self, x):
    return self.net(x)

    pdeN = pdenet()#intance
```

The B function connects the two networks and ensures that the four boundaries pass into B as 0.

```
    def B(x, y):
    B = x * (1 - x) * y * (1 - y)
    return B
```

3.1.3 Creat a class to solve PDE

1. init:
    - Parameters: epochs, N_b (number of boundary points), $N_f(numberofinteriorpoints)$, and h (grid size)
    - Call instance for networks and use MSE loss to train.
    - Compute reference solution on the grid.
2. Set the exact solution and rhs (to be polulated).
3. Data Preparation:
    - Boundary data: 'up, down, left, right' function is created to generate data along 4 sides and compute the boundary condition, return the 'requires gradient True' for x and y to train later.
    - Inner domain data: Coordinates are generated inside the unit square and feed into rhs to compute 2.
    - Reference points: Grid points is obtained by the gridpoint function to record l2 error 8 as well as comparison.
4. Compute loss:
    - Boundary loss: Obtain x and y values on boundary ⇒ Use 'torch.cat' to stack x and y into coordinates ⇒ Feed the points into the boundary network to obtain the predicted value on boundary ⇒ Call MSE method (self.loss) to compute loss on boundary.
    - PDE loss:
        - Prepare gradient function: compute gradient recursively.

    - For order 1, 'torch.autograd.grad': Compute the first-order gradient of 'u' with respect to 'x'; 'ones like': Create a tensor of ones with the same size as 'u' to ensure correct dimensions for computing the gradient; 'create graph = True': Allow higher-order gradients to be computed; 'only inputs = True': Specify that no other variables depend on 'u'; '[0]': Return the first element of the resulting gradient tuple.
    - For order 2, 'self.gradients(u, x)': Compute the first-order derivative of u with respect to x. The result is passed as the 'u' parameter under 'order = 1' condition to compute the gradient again to obtain the second-order derivative.
  - Obtain x and y values on inner domain ⇒ Use 'torch.cat' to stack x and y into coordinates ⇒ Feed the points into the PDE network and call gradient function to compute Δu_h to obtain the predicted value on PDE ⇒ Call MSE method (self.loss) to compute loss within domain.
- l2 error: Compute the L2 error between the predicted solution and the real solution using 'torch.sqrt'.

```
class PDE_solver:
def __init__(self, epochs, N_b, N_f, h):
self.epochs = epochs
self.N_b = N_b
self.N_f = N_f
self.h = h

self.u_b = boundaryN
self.u_pde = pdeN
self.loss = torch.nn.MSELoss()

xm, ym, xx, yy, xy = self.gridpoint()
self.u = self.exactsol(xx, yy)

'''Functions to feed'''
def exactsol(self, x, y):
raise NotImplementedError
def rhs(self, x, y):
raise NotImplementedError

'''Data preparation: Boundary, Interior, Reference points '''

#Boundary data
def up(self):  # boundary1 u(x,1)=0
x = torch.rand(self.N_b, 1)
y = torch.ones_like(x)
```

```
    b_up = self.exactsol(x, y)
    return x.requires_grad_(True), y.requires_grad_(True), b_up

    def down(self):  # boundary2 u(x, 0)=0
    x = torch.rand(self.N_b, 1)
    y = torch.zeros_like(x)  # the same size as x
    b_down = self.exactsol(x, y)
    return x.requires_grad_(True), y.requires_grad_(True), b_down

    def left(self):  # boundary3 u(0, y)=0
    y = torch.rand(self.N_b, 1)
    x = torch.zeros_like(y)
    b_left = self.exactsol(x, y)
    return x.requires_grad_(True), y.requires_grad_(True), b_left

    def right(self):  # boundary4 u(1, y)=0
    y = torch.rand(self.N_b, 1)
    x = torch.ones_like(y)
    b_right = self.exactsol(x, y)
    return x.requires_grad_(True), y.requires_grad_(True), b_right

    #Inner data
    def interior(self):
    x = torch.rand(self.N_f, 1)
    y = torch.rand(self.N_f, 1)
    f = self.rhs(x, y)
    return x.requires_grad_(True), y.requires_grad_(True), f

    #reference points
    def gridpoint(self):
    xc = torch.linspace(0, 1, self.h)
    xm, ym = torch.meshgrid(xc, xc, indexing='ij')
    xx = xm.reshape(-1, 1)
    yy = ym.reshape(-1, 1)
    xy = torch.cat([xx, yy], dim=1)
    return xm, ym, xx, yy, xy

    '''Compute loss: boundary loss, gradient, pde loss, l2error '''

    def bloss(self):
    xu, yu, b_up = self.up()
    xd, yd, b_down = self.down()
    xl, yl, b_left = self.left()
```

```
    xr, yr, b_right = self.right()
    ub = self.u_b(torch.cat([xd, yd], dim=1)) + self.u_b(torch.cat([
       xu, yu], dim=1)) + self.u_b(torch.cat([xl, yl], dim=1)) + self
       .u_b(torch.cat([xr, yr], dim=1))
    breal = b_up + b_down + b_left + b_right
    return self.loss(ub, breal)

    def gradients(self, u, x, order = 1):
    if order == 1:
    return torch.autograd.grad(u, x, grad_outputs=torch.ones_like(u),
        create_graph=True, only_inputs=True, )[0]
    else:
    return self.gradients(self.gradients(u, x), x, order = order -1)

    def pdeloss(self):
    x, y, f = self.interior()
    uh = self.u_b(torch.cat([x, y], dim=1)) + B(x, y) * self.u_pde(
    torch.cat([x, y], dim=1))  # usetorch.cat to connect x and y as
       the column method
    delta_uh = self.gradients(uh, x, 2) + self.gradients(uh, y, 2)
    return self.loss(delta_uh, f)  # delta u_h-delta u

    def l2_error(self, u_pred, u_real):
    l2 = torch.sqrt(self.loss(u_pred, u_real))
    return l2
```

3.1.4 Create a class for the example

1. Inherit the parent class PDE solver and feed the exact and rhs functions.
2. Pass parameter into the parent class.

```
    class wellbehaved(PDE_solver):
    def exactsol(self, x, y):
    u = torch.sin(torch.pi * x) * torch.sin(torch.pi * y)
    return u

    def rhs(self, x, y):
    f = -2 * torch.pi ** 2 * torch.sin(torch.pi * x) * torch.sin(
       torch.pi * y)
    return f

example1 = wellbehaved(1400, 200, 1000, 50)
```

3.1.5 Training and Drawing

1. Training: with parameters of epochs, h and epsilon

   - Using Adam optimizer to update the neural network.
   - Initialize 3 empty lists to store the loss values later.
   - For each iteration: Train the boundary network in k steps and stop updating until it reaches the threshold of 10^{-5} and record the 'bloss' ⇒ Train the PDE network and record the 'ploss' ⇒ Compute the trained neural network and the exact solution on the grid ⇒ Show progress of train ⇒ Return losses and u_h to draw

```
        def train(epochs, k, epsilon):
        bloss = []
        ploss = []
        l2 = []

        opt_b = torch.optim.Adam(params=boundaryN.parameters())
        opt_p = torch.optim.Adam(params=pdeN.parameters())

        for i in range(epochs):

        '''train boundary network for k steps, record bloss'''
        for _ in range(k):
        opt_b.zero_grad()  # Zero out
        lb = example1.bloss()
        if lb.item() > epsilon:# extract a single element value
        lb.backward()  # backwardpropagation
        opt_b.step()  # update the parameter
        else:
        break
        bloss.append(lb.item())

        '''train pde network, record ploss'''
        opt_p.zero_grad()
        lp = example1.pdeloss()
        lp.backward()
        opt_p.step()
        ploss.append(lp.item())

        '''record l2 error'''
        xm, ym, xx, yy, xy = example1.gridpoint()
        u_pred = example1.u_b(xy) + B(xx, yy) * example1.u_pde(xy
           )
```

```
        u_real = example1.u
        l2.append(example1.l2_error(u_pred, u_real).item())

        if i % 10 == 0:
        print('i= ', i, 'pdeloss: ', lp, '\n', 'bloss: ', lb)
        return bloss, ploss, l2, u_pred

bloss, ploss, l2, u_pred = train(example1.epochs, 5, 1e-5)
```

2. Draw: with parameters of epochs and h

   - Comparison of numerical solution by neural networks and exact solution: Creating a list of iterations ⇒ Extracting grid points ⇒ Reshaping and converting the solution arrays ⇒ Creating subplots for the analytical and numerical solution respectively
   - Record 3 losses: create 3 separe plots and set xscale('linear') while yscale('log') to visualize clearly.
   - Save figures.

```
        def draw(epochs, h):
        iterations_list = [i for i in range(1, epochs+1)]
        xm, ym, xx, yy, xy = example1.gridpoint()

        '''Numerical and Analytical'''
        u_pred_fig = u_pred.reshape(h, h).detach().numpy()
        u_real_fig = example1.u.reshape(h, h).detach().numpy()
        fig = plt.figure()
        ax = fig.add_subplot(111, projection='3d')
        ax.plot_surface(xm, ym, u_real_fig, cmap='Blues')
        ax.plot_surface(xm, ym, u_pred_fig, cmap='Reds')
        ax.set_xlabel('X')
        ax.set_ylabel('Y')
        ax.set_zlabel('Z')

        '''Record loss'''
        fig1 = plt.figure()
        fig2 = plt.figure()
        fig3 = plt.figure()

        # boundary loss
        ax1 = fig1.add_subplot(111)
        ax1.plot(iterations_list, bloss, 'r-+', linewidth=2,
           label='Bloss')
        ax1.set_xlabel('Iterations')
```

```
        ax1.set_ylabel('Boundary Loss')
        ax1.set_xscale('linear')
        ax1.set_yscale('log')
        ax1.legend()

        # pde loss
        ax2 = fig2.add_subplot(111)
        ax2.plot(iterations_list, ploss, 'g-+', linewidth=2,
           label='PDE Loss')
        ax2.set_xlabel('Iterations')
        ax2.set_ylabel('PDE Loss')
        ax2.set_xscale('linear')
        ax2.set_yscale('log')
        ax2.legend()

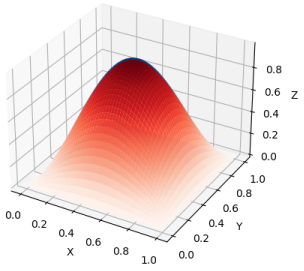
        # l2 error
        ax3 = fig3.add_subplot(111)
        ax3.plot(iterations_list, l2, 'b-+', linewidth=2, label='
           L2 Error')
        ax3.set_xlabel('Iterations')
        ax3.set_ylabel('L2 Error')
        ax3.set_xscale('linear')
        ax3.set_yscale('log')
        ax3.legend()

        fig1.savefig('Boundary_Loss_for_Well_Behaved_Solution.pdf
           ')
        fig2.savefig('PDE_Loss_for_Well_Behaved_Solution.pdf')
        fig3.savefig('L2_Error_for_well_Behaved_Solution.pdf')
        plt.show()
draw(example1.epochs, example1.h)
```

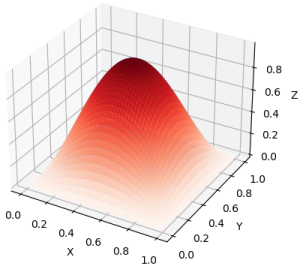
4 Analysis of The Result

1. Comparison: Blue represents the true solution and red represents the numerical solution. It is difficult to distinguish between the true and numerical solutions until around 600 iterations in figure 2a; After 1400 iterations in figure 2b, it becomes challenging to differentiate between exact and numerical solutions due to both the PDE loss 4 and L2 error 5 reaching 10^{-3}.

2. Boundary loss: During the update process, the boundary loss experiences a dramatic drop, after which it remains stable at an error rate of approximately 10^{-5} due to the early termination shown in figure 3. To ensure smooth updates to the boundary network and maintain



(a) Comparison (Iterations = 600, Blue= exact solution, Red = numerical solution)



(b) Comparison (Iterations = 1400, Blue= exact solution, Red = numerical solution)

the loss function at an appropriate level, this setting may be implemented. Otherwise, the loss function may become too small to be properly updated and could result in issues with backward propagation.

3. PDE loss: The PDE loss does not effectively drop until the iteration range of 400-600, indicating early inefficacy. However, it decreases thereafter and approaches 10^{-3} after 1400 iterations in figure 4.

4. L_2 error: L2 error drops significantly as update begins, following similar trend as PDE loss, and stabilizes at 10^{-3} after 1000 iterations in figure 5.

After 600 iterations, the model is able to effectively approximate $\sin \pi x \sin \pi y$ based on the three metric indicators. However, it faces difficulty in reducing the PDE loss in some extreme conditions, such as curves with peaks and less regular curves, which suggests a certain degree of limitation.

5 Reflection of Code

1. To ensure code reusability, define functions and classes, modifying only necessary items.

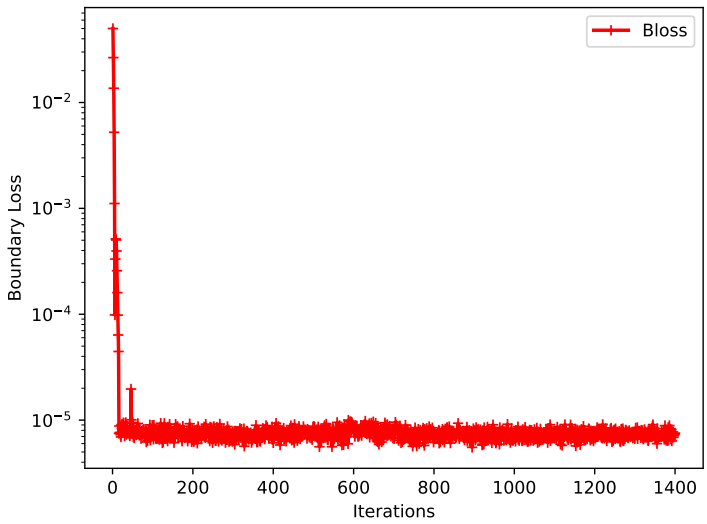


Figure 3: Boundary loss ($N_b = 200$, iteration = 1400)

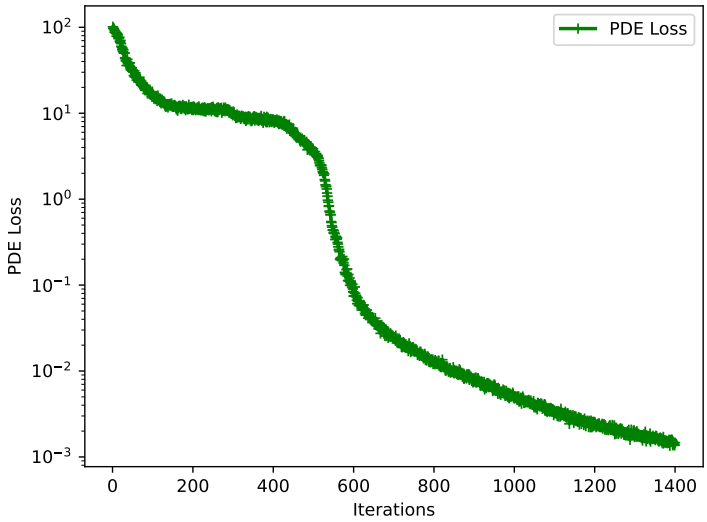


Figure 4: PDE loss ($N_f = 1000$, iteration = 1400)

2. When calculating gradients,

   - It is necessary to focus on variables rather than specific coordinates otherwise may return None.
   - Separate x and y as the gradient of u can only be computed concerning one variable while connect the coordinates for passing into Neural Networks.

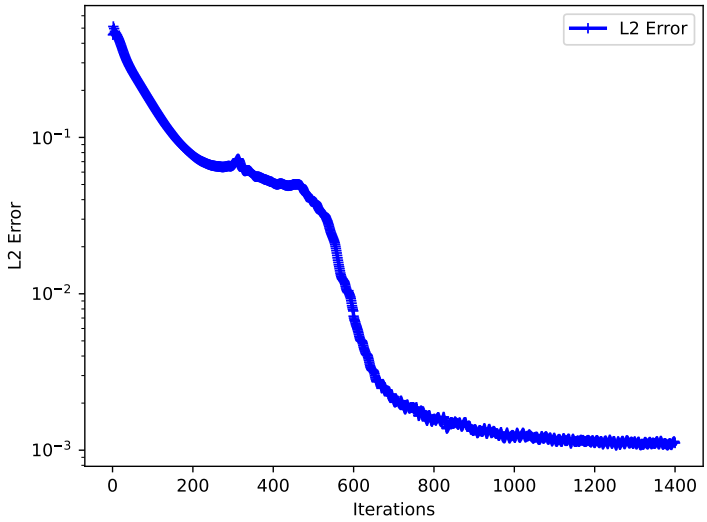


Figure 5: L2 error (iteration = 1400)

3. It is crucial to establish the structure before beginning the implementation of code, which may make it more organized and efficient.

6 Conclusion

In the report, a method is described that uses two neural networks to solve the Poisson equation 1. The method is inspired by a journal and employs training algorithms similar to GAN: one neural network approximates the boundary while the other approximates the PDE. The boundary network is updated several times in each iteration before the PDE network is updated once. This compensates for the fewer boundary training points compared to the points within the domain. However, the method has limitations in peak and less regular solutions, and its fitting ability is demonstrated only in some well-behaved solutions such as the example mentioned above. It has limitations in peak and less regular solutions and needs further improvements.

References

1. Deep learning for Partial Differential Equations (PDEs)

2. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations